Experiment No. 1

Real-World Data Collection, Cleaning and Preprocessing using Python

Aim

To collect a real-world dataset (Titanic Survival Dataset) from a public source and perform data preprocessing by handling missing values, duplicate records, inconsistent formats, categorical variables and outliers using Python.

Requirements

- Python 3, Jupyter Notebook / Google Colab
- Libraries: Pandas, NumPy, Matplotlib, Seaborn, scikit-learn
- Dataset: Titanic (Kaggle *train.csv*) - 891 passengers, 12 columns. Target: *Survived*.

Procedure

- Download the Titanic dataset from Kaggle (*train.csv*) and place it next to the notebook.
- Import libraries, load the data and inspect it with `head()`, `info()`, `describe()`.
- Find and treat missing values (median / mode / drop column).
- Find and remove duplicate rows; standardise text categories.
- Encode categorical variables (Label and One-Hot encoding).
- Detect outliers with box plots and the IQR rule, then treat them (capping).
- Scale numeric features (standardisation and min-max normalisation).
- Create new features (FamilySize, IsAlone, AgeGroup, Title).
- Save the cleaned dataset as *titanic_cleaned.csv*.

Program and Output

The program below was run on the actual Titanic dataset; the green boxes show the real output obtained.

Step 2-3: Import libraries, load and inspect the data

```
import pandas as pd
import numpy as np
import matplotlib
matplotlib.use("Agg")                      # remove this line in Jupyter/Colab
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.preprocessing import MinMaxScaler, StandardScaler

pd.set_option("display.width", 120)
pd.set_option("display.max_columns", 20)

# ---------- Step 3: load and inspect ----------
df = pd.read_csv("titanic.csv")            # Kaggle: train.csv
print("Shape:", df.shape)
print(df.head(3)[["PassengerId", "Survived", "Pclass", "Sex", "Age", "Fare", "Embarked"]])
df.info()
print(df.describe().round(2).T)
```

Output:

```
Shape: (891, 12)
   PassengerId  Survived  Pclass     Sex   Age     Fare Embarked
0            1         0       3    male  22.0   7.2500        S
1            2         1       1  female  38.0  71.2833        C
2            3         1       3  female  26.0   7.9250        S
<class 'pandas.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 12 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
```

```
 0   PassengerId  891 non-null    int64
 1   Survived     891 non-null    int64
 2   Pclass       891 non-null    int64
 3   Name         891 non-null    str
 4   Sex          891 non-null    str
 5   Age          714 non-null    float64
 6   SibSp        891 non-null    int64
 7   Parch        891 non-null    int64
 8   Ticket       891 non-null    str
 9   Fare         891 non-null    float64
 10  Cabin        204 non-null    str
 11  Embarked     889 non-null    str
dtypes: float64(2), int64(5), str(5)
memory usage: 83.7 KB
             count    mean     std   min     25%     50%    75%     max
PassengerId  891.0  446.00  257.35  1.00  223.50  446.00  668.5  891.00
Survived     891.0    0.38    0.49  0.00    0.00    0.00    1.0    1.00
Pclass       891.0    2.31    0.84  1.00    2.00    3.00    3.0    3.00
Age          714.0   29.70   14.53  0.42   20.12   28.00   38.0   80.00
SibSp        891.0    0.52    1.10  0.00    0.00    0.00    1.0    8.00
Parch        891.0    0.38    0.81  0.00    0.00    0.00    0.0    6.00
Fare         891.0   32.20   49.69  0.00    7.91   14.45   31.0  512.33
```

Step 4: Handle missing values

```
# ---------- Step 4: missing values ----------
print("\nMissing values:\n", df.isnull().sum()[df.isnull().sum() > 0])
print("Cabin missing %:", round(df["Cabin"].isnull().mean() * 100, 1))

df["Has_Cabin"] = df["Cabin"].notnull().astype(int)   # keep the information
df = df.drop(columns="Cabin")                         # ~77% missing -> drop column
df["Age"] = df["Age"].fillna(df["Age"].median())      # numeric -> median
df["Embarked"] = df["Embarked"].fillna(df["Embarked"].mode()[0])  # categorical -> mode
print("Missing after treatment:", int(df.isnull().sum().sum()))
```

Output:

```
Missing values:
 Age         177
Cabin       687
Embarked      2
dtype: int64
Cabin missing %: 77.1
Missing after treatment: 0
```

Step 5: Remove duplicate records

```
# ---------- Step 5: duplicates ----------
print("\nDuplicate rows:", df.duplicated().sum())
df = df.drop_duplicates()
print("Duplicate rows after removal:", df.duplicated().sum(), "| Shape:", df.shape)
```

Output:

```
Duplicate rows: 0
Duplicate rows after removal: 0 | Shape: (891, 12)
```

Step 6: Standardise and encode categorical data

```
# ---------- Step 6: standardise + encode categorical data ----------
df["Sex"] = df["Sex"].str.strip().str.lower()
df["Embarked"] = df["Embarked"].str.strip().str.upper()
print("\nSex values:", list(df["Sex"].unique()), "| Embarked values:", list(df["Embarked"].unique()))

df["Sex"] = df["Sex"].map({"male": 0, "female": 1})   # label encoding (binary)
df = pd.get_dummies(df, columns=["Embarked"], drop_first=True, dtype=int)  # one-hot
```

Output:

```
Sex values: ['male', 'female'] | Embarked values: ['S', 'C', 'Q']
```

Step 7: Detect and treat outliers (Box plot + IQR)

```
# ---------- Step 7: outliers (box plot + IQR) ----------
cols = ["Age", "Fare"]
fig, ax = plt.subplots(2, 2, figsize=(10, 6))
for i, c in enumerate(cols):
    sns.boxplot(x=df[c], ax=ax[0, i], color="salmon"); ax[0, i].set_title(f"{c} - before")

for c in cols:
    q1, q3 = df[c].quantile([0.25, 0.75])
    iqr = q3 - q1
```

```
    low, high = q1 - 1.5 * iqr, q3 + 1.5 * iqr
    n_out = ((df[c] < low) | (df[c] > high)).sum()
    print(f"{c}: Q1={q1:.2f} Q3={q3:.2f} IQR={iqr:.2f} bounds=({low:.2f}, {high:.2f}) outliers={n_out}")
    df[c] = df[c].clip(low, high)                    # cap (winsorize)

for i, c in enumerate(cols):
    sns.boxplot(x=df[c], ax=ax[1, i], color="lightgreen"); ax[1, i].set_title(f"{c} - after")
plt.tight_layout(); plt.savefig("boxplots.png", dpi=110); plt.show()
```

Output:

```
Age: Q1=22.00 Q3=35.00 IQR=13.00 bounds=(2.50, 54.50) outliers=66
Fare: Q1=7.91 Q3=31.00 IQR=23.09 bounds=(-26.72, 65.63) outliers=116
```

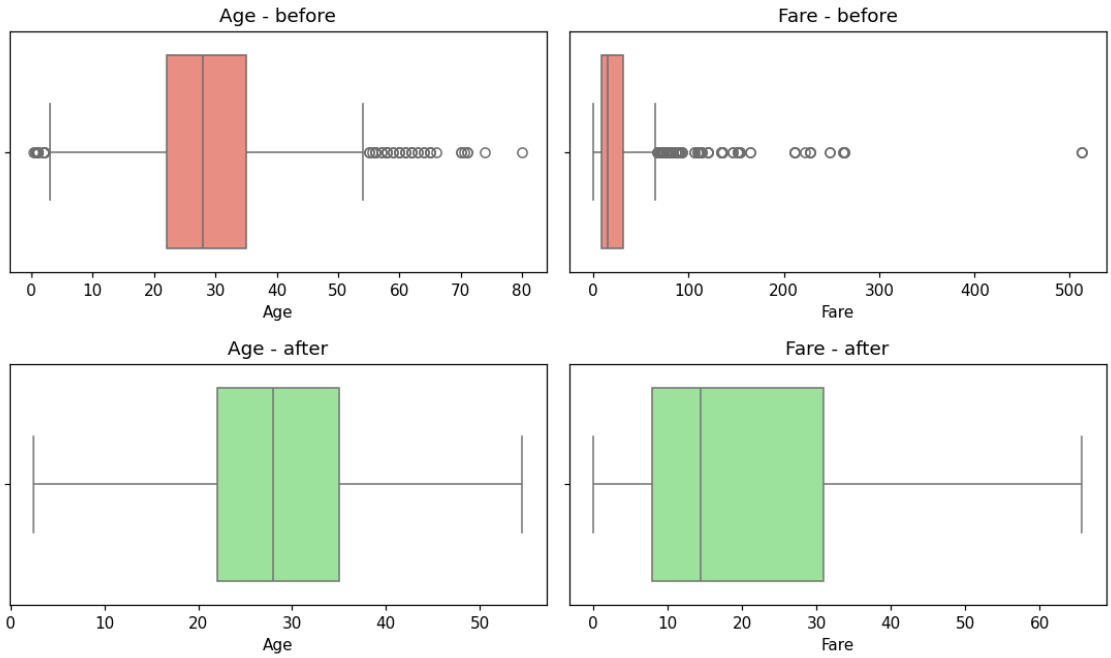

Fig. 1: Box plots of Age and Fare before (top) and after (bottom) IQR capping.

Step 8: Normalisation / Standardisation

```
# ---------- Step 8: scaling ----------
df[["Age_std", "Fare_std"]] = StandardScaler().fit_transform(df[["Age", "Fare"]])
df[["Age_norm", "Fare_norm"]] = MinMaxScaler().fit_transform(df[["Age", "Fare"]])
print("\n", df[["Age_std", "Fare_std", "Age_norm", "Fare_norm"]].describe().loc[["mean", "std", "min", "max"]].round(3)
```

Output:

```
       Age_std  Fare_std  Age_norm  Fare_norm
mean     0.000     0.000     0.510      0.366
std      1.001     1.001     0.232      0.312
min     -2.200    -1.175     0.000      0.000
max      2.110     2.032     1.000      1.000
```

Step 9: Feature engineering

```
# ---------- Step 9: feature engineering ----------
df["FamilySize"] = df["SibSp"] + df["Parch"] + 1
df["IsAlone"] = (df["FamilySize"] == 1).astype(int)
df["AgeGroup"] = pd.cut(df["Age"], bins=[0, 12, 18, 35, 60, 100],
                        labels=["Child", "Teen", "YoungAdult", "Adult", "Senior"])
df["Title"] = df["Name"].str.extract(r" ([A-Za-z]+)\.", expand=False)
df["Title"] = df["Title"].replace({"Mlle": "Miss", "Ms": "Miss", "Mme": "Mrs"})
common = ["Mr", "Mrs", "Miss", "Master"]
df["Title"] = df["Title"].where(df["Title"].isin(common), "Rare")
print("\nTitle counts:\n", df["Title"].value_counts())

df = df.drop(columns=["Name", "Ticket"])              # high-cardinality text columns
df = pd.get_dummies(df, columns=["Title", "AgeGroup"], dtype=int)
```

Output:

```
Title counts:
 Title
Mr        517
Miss      185
```

```
Mrs       126
Master     40
Rare       23
Name: count, dtype: int64
```

Step 10: Save the cleaned dataset

```
# ---------- Step 10: save ----------
df.to_csv("titanic_cleaned.csv", index=False)
print("\nSaved titanic_cleaned.csv | Final shape:", df.shape)
print(list(df.columns))
```

Output:

```
Saved titanic_cleaned.csv | Final shape: (891, 27)
['PassengerId', 'Survived', 'Pclass', 'Sex', 'Age', 'SibSp', 'Parch', 'Fare', 'Has_Cabin', 'Embarked_Q', 'Embarked_S', 'Age
```

Observations and Result

Aspect	Finding
Dataset size	891 rows x 12 columns originally; 891 rows x 27 columns after preprocessing
Missing values	Age: 177 (filled with median 28), Embarked: 2 (filled with mode), Cabin: 687 = 77.1% (dropped; Has_Cabin flag kept)
Duplicates	0 duplicate rows found in this dataset (check performed and drop_duplicates applied)
Categorical encoding	Sex: label encoded (male=0, female=1); Embarked, Title, AgeGroup: one-hot encoded
Outliers (IQR)	Age: 66 outliers (bounds 2.5-54.5); Fare: 116 outliers (upper bound 65.63); capped to the bounds
Scaling	Standardised columns have mean 0, SD 1; min-max columns lie in [0, 1]
New features	FamilySize, IsAlone, AgeGroup, Title, Has_Cabin
Output file	titanic_cleaned.csv (no missing values, all numeric)

Result: The raw Titanic data was successfully cleaned and transformed into a complete, numeric, scaled dataset that is ready for visualisation and machine learning.

Answers to Questions

Q1. Why is data preprocessing one of the most important phases in data analytics?

Real-world data is incomplete, inconsistent, duplicated and noisy. Analysis or models built on such data give unreliable results ("garbage in, garbage out"). Preprocessing:

- improves data quality and accuracy, so conclusions are trustworthy;
- makes data suitable for algorithms, which cannot handle missing values or text categories directly;
- reduces bias and noise caused by outliers, duplicates and wrong formats;
- improves model performance and training speed (e.g. through scaling and feature selection);
- typically takes 60-80% of a data scientist's time because it decides the quality of every later step.

Q2. Explain different methods of handling missing values with examples.

Method	Description	Example
Deletion (rows)	Remove records with missing values; fine when few rows are affected	Drop the 2 Titanic rows with missing Embarked
Deletion (columns)	Remove a feature with too many missing values	Drop Cabin (77% missing)
Mean imputation	Replace with column mean; for symmetric numeric data	Age = mean age
Median imputation	Replace with median; robust to outliers/skew	Age = 28, Fare, Income
Mode imputation	Replace with most frequent value; for categorical data	Embarked = 'S'
Constant / indicator	Fill with a constant (e.g. 'Unknown') or add a flag column	Has_Cabin = 1 if Cabin present
Forward / backward fill	Copy previous/next value; for time series	Sensor or stock readings
Model-based	Predict missing values with KNN, regression or iterative imputers	KNNImputer for Age using Pclass, Fare

Q3. Differentiate between Label Encoding and One-Hot Encoding.

Aspect	Label Encoding	One-Hot Encoding
Method	Assigns one integer per category	Creates one binary (0/1) column per category
Example	Low=0, Medium=1, High=2; male=0, female=1	Embarked to Embarked_C, Embarked_Q, Embarked_S
Columns	Single column	Number of categories (or n-1)
Order implied?	Yes - models may assume 2 > 1 > 0	No - categories treated as equal
Best for	Ordinal data, binary variables, tree models	Nominal (unordered) data like city, department
Drawback	Creates false ranking for nominal data	Many columns for high-cardinality features

Q4. What are outliers? How can they affect analytical results?

Outliers are data points that differ markedly from the rest of the observations, caused by measurement/entry errors or by genuine extreme cases (e.g. a Titanic fare of 512 when the median is 14.45). They can:

- distort the mean, standard deviation and correlations;
- skew regression lines and reduce the accuracy of linear/distance-based models;
- stretch the scale in normalisation, squeezing normal values into a tiny range;

- lead to wrong business decisions. Treatment: capping/winsorising, transformation (log), removal if they are errors, or keeping them if they are genuine.

IQR rule: a value is an outlier if it is below Q1 - 1.5 x IQR or above Q3 + 1.5 x IQR.

Q5. Difference between normalization and standardization.

Aspect	Normalization (Min-Max)	Standardization (Z-score)
Formula	x' = (x - min) / (max - min)	z = (x - mean) / std
Result range	0 to 1 (or chosen range)	No fixed range; mean 0, SD 1
Outlier sensitivity	High - min/max change the scale	Lower, but still affected
Use when	Distribution unknown/non-Gaussian; neural networks, KNN, image data	Data roughly Gaussian; SVM, logistic/linear regression, PCA
sklearn	MinMaxScaler	StandardScaler

Q6. Why should duplicate records be removed before analysis?

- They **inflate counts and statistics** (means, totals, frequencies), giving biased results.
- In machine learning they cause **data leakage** when the same record falls in both training and test sets, producing falsely high accuracy.
- They make the model overfit to repeated patterns and waste storage and computation.
- They indicate data-collection or merging errors that should be fixed for consistency.

Q7. What is feature engineering? Give two practical examples.

Feature engineering is the process of creating, transforming or selecting input variables using domain knowledge so that machine learning models can learn better patterns.

- **Titanic:** FamilySize = SibSp + Parch + 1, and IsAlone flag; Age binned into AgeGroup (Child, Teen, ...); Title extracted from Name (Mr, Mrs, Miss, Master).
- **IBM HR:** IncomeCategory (Low/Medium/High) from MonthlyIncome; YearsSincePromotion ratio; tenure = YearsAtCompany / TotalWorkingYears.
- **Other:** extracting day/month/weekday from a date; BMI = weight / height squared.

Q8. Which preprocessing techniques would you apply to the IBM HR Employee Attrition dataset and why?

Technique	Reason
Inspect and check missing values / duplicates	The dataset (about 1,470 rows, 35 columns) is generally clean, but this must be verified.
Drop useless columns: EmployeeCount, Over18, StandardHours, EmployeeNumber	They are constant or just IDs and carry no predictive information.
Encode target Attrition (Yes/No to 1/0)	Needed for classification.
Label/ordinal encoding: OverTime, Gender, BusinessTravel	Binary or ordered categories.
One-hot encoding: Department, JobRole, MaritalStatus, EducationField	Nominal categories without order.
Outlier handling (IQR / capping / log transform) on MonthlyIncome, YearsAtCompany, TotalWorkingYears	These are right-skewed with extreme values.
Scaling (StandardScaler / MinMaxScaler)	Features such as Age, DailyRate, MonthlyIncome are on very different scales.

Feature engineering: IncomeCategory, AgeGroup, promotion/tenure ratios	Captures patterns such as low income plus overtime leading to attrition.
Handle class imbalance (about 16% 'Yes'): SMOTE, class weights, stratified split	Otherwise the model predicts 'No' for everyone and gets high accuracy but is useless.

Q9. How does poor-quality data affect machine learning model performance?

- **Low accuracy:** noise, wrong labels and missing values hide the true patterns.
- **Bias:** unrepresentative or imbalanced data produces unfair or skewed predictions.
- **Overfitting / poor generalisation:** the model learns errors and duplicates instead of real relationships.
- **Training problems:** unscaled features and outliers slow or destabilise convergence; algorithms may fail on NaN values.
- **Unreliable decisions and higher cost** in deployment (wrong diagnoses, loan decisions, etc.).

Q10. Name three Python libraries commonly used for data preprocessing.

- **Pandas** - loading, cleaning, merging, handling missing values and duplicates, encoding (get_dummies).
- **NumPy** - numerical arrays, mathematical and statistical operations.
- **Scikit-learn** - SimpleImputer, StandardScaler, MinMaxScaler, LabelEncoder, OneHotEncoder, train_test_split.
- (Also: Matplotlib/Seaborn for visual checks, SciPy for statistics, imbalanced-learn for SMOTE.)

Conclusion

Real-world Titanic data was collected, inspected and preprocessed using Python. Missing values were imputed or dropped, duplicates were checked, categorical variables were encoded, outliers were detected with box plots and the IQR method and capped, numerical attributes were scaled and new features were engineered. The final dataset (891 x 27) contains no missing values and was saved as *titanic_cleaned.csv*, ready for visualisation and machine learning.